LAB 7

Fuzzing for Input Validation Bugs

Student	Pranay Shah
SRN	PES2UG24CS366
Email	pranay.vshah06@gmail.com
GitHub	PurposeKnight
Repository	PurposeKnight/L6_L7_58_restaurant_bill
Tools	Python 3, pytest, Hypothesis

Objective: use property-based fuzzing to find edge-case input validation bugs in all three restaurant functions, then harden the implementation and verify the corrected behavior.

1. Scope and method

The assigned repository contained the three-function restaurant application from Lab 6 but no separate Lab 7 starter folder. Therefore, the same three functions—bill_total, split_bill, and table_wait—were used as the processor targets. The final implementation is stored as processor.py under LAB7.

The Hypothesis suite generates valid and malformed values for every input boundary. Each property runs 150–250 examples and checks reference arithmetic, output invariants, domain limits, malformed types, Unicode text, None, NaN, negative values, large integers, and the special bool-is-an-int edge case.

2. Bugs discovered

Counterexample	Observed defect	Correction
food_total=None	Incidental TypeError crash	Explicit integer validation
people=None	Incidental TypeError crash	Positive integer validation
total=-1	Negative bill accepted	Require total >= 0
tip_percent=101	Unbounded tip accepted	Require 0..100
party_size=None	Incidental TypeError crash	Validate all table inputs
bool as numeric input	Accepted because bool subclasses int	Use exact-type checks

3. Step 2 — Run the buggy application

The baseline succeeds for ordinary sample values, which explains why conventional happy-path testing did not reveal the validation defects.

```
PS ...\LAB7> python processor.py
Restaurant Processor - Buggy Baseline
Dine-in total: 924
Per-person share: 254
Table decision: waitlist
```

4. Step 4 — Failing Hypothesis tests

The property-based suite automatically finds invalid inputs that crash or are incorrectly accepted by the baseline implementation.

```
PS ...\LAB7> python -m pytest test_processor.py -q --tb=no
..F.FF.FFFF..F                                                       [100%]
========================== short test summary info ==========================
FAILED test_processor.py::test_bill_total_rejects_non_integer_money - BaseExc...
FAILED test_processor.py::test_bill_total_rejects_invalid_guests_or_flag - Ty...
FAILED test_processor.py::test_split_bill_rejects_invalid_people - BaseExcept...
FAILED test_processor.py::test_split_bill_rejects_negative_total - Failed: DI...
FAILED test_processor.py::test_split_bill_rejects_tip_outside_percentage_range
FAILED test_processor.py::test_split_bill_rejects_non_integer_total - BaseExc...
FAILED test_processor.py::test_split_bill_rejects_non_integer_tip - BaseExcep...
FAILED test_processor.py::test_table_wait_rejects_malformed_inputs - BaseExce...
8 failed, 6 passed in 8.45s
```

5. Step 5 — Run the fixed application

Normal outputs are preserved, while an invalid negative total is now rejected with a predictable ValueError message.

```
PS ...\LAB7> python processor.py
Restaurant Processor - Validated Version
Dine-in total: 924
Per-person share: 254
Table decision: waitlist
Invalid input rejected: total must be >= 0
```

6. Step 6 — Re-run tests after the fix

All 14 properties pass after adding strict, centralized validation and explicit domain bounds.

```
PS ...\LAB7> python -m pytest test_processor.py -q
..............                                                           [100%]
14 passed in 1.71s
```

Conclusion

Hypothesis exposed failures that ordinary examples missed and reduced them to actionable counterexamples. The corrected processor now rejects malformed input consistently with ValueError, preserves correct restaurant calculations, and passes the complete property-based fuzz suite.